FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES
FAST School of Computing · Karachi Campus · Department of Artificial Intelligence & Data Science
AL-2002 Artificial Intelligence (Laboratory) · Fall 2026

PROJECT PROPOSAL

Semester Lab Project · Weightage 1 mark

Group Information

	Name	Roll Number	Section	Role in the project
Leader	Sameer Danish	24K-2504	BDS-5B	Team leader; machine-learning models and ensemble
Member 2	Hadeer Abbas	24K-2503	BDS-5B	Data pipeline, feature extraction and evaluation
Member 3	Muhammad Arham	24K-2529	BDS-5B	Back end, SHAP explanations and web interface

Project Title	PhishGuard AI: Real-Time Phishing URL Detector with Explainable Predictions
Project Idea	Idea No. _____ from the Project Ideas list ☐ Own idea (needs approval) ☑
Category	Part A — Game ☐ Part B — Applied / Data ☑
GitHub Repository URL	https://github.com/MuhammadArham13/AI-Project.git
Date of Submission	First week of December 2026 (Week 15)

1. Project Title and Summary

Title: PhishGuard AI: Real-Time Phishing URL Detector with Explainable Predictions

PhishGuard AI is a small web application that tells a person, within a few seconds, whether a web link looks safe or like a phishing attempt (a fake page that steals passwords or bank details). The user pastes a URL and receives a Safe/Phishing verdict, a probability score and a short plain-language list of the reasons behind it, such as an '@' symbol hiding the real host or an unusually long address. It is meant for everyday internet users, students and junior security analysts who need a quick second opinion before clicking. AI makes this possible because the system learns what phishing links look like from thousands of labelled examples, so it can flag brand-new links that no blacklist has recorded yet, which a plain lookup program cannot do. Two learned models (one reading numeric features, one reading the raw characters) are combined, and an explanation method shows which parts of the URL pushed the decision. Everything runs locally, with no paid service.

2. Problem Statement and Motivation

Phishing is one of the most common ways attackers steal passwords and bank details: the victim receives a link that imitates a trusted site and is tricked into typing private information. The space of possible URLs is effectively unlimited, because a hostname, path and query can be any mix of letters, digits and symbols, so no list can ever contain every malicious address and checking links by hand is impractical. Attackers also disguise links with an '@' that hides the real host, digits in place of letters (paypa1), long chains of subdomains, hexadecimal IP addresses and URL shorteners. Ordinary users cope today by eyeballing the link or relying on blacklists, but a blacklist only knows sites that were already reported, so a brand-new (zero-day) phishing page goes unnoticed until victims have been harmed. A model that learns the structure of phishing URLs can judge a link it has never seen, instantly. Solving this is worthwhile because it protects users at the moment of the click, before any damage is done.

3. Problem Formulation

3.1 PEAS Description

Component	Description for your project
Performance measure	Accuracy, precision, recall and F1 on a held-out test set (target: accuracy of about 85% or more, with few missed phishing URLs); response time under 3 seconds per URL; each verdict shown with its top reasons.
Environment	A stream of URL strings submitted through a web page, plus a labelled dataset of legitimate and phishing URLs used for training; everything runs on the user's own computer.

Component	Description for your project
Actuators	The web page and API response: Safe/Phishing verdict, probability bar and plain-language reasons (SHAP-based); a warning message when the URL is flagged.
Sensors	The URL text typed or pasted by the user, parsed into scheme, host, subdomains, path and symbols; the same text as a sequence of characters.

3.2 Environment and Agent Type

Property	Classification — tick one
Observability	☐ Fully observable ☑ Partially observable *(only the URL text is seen, not the page or its owner)*
Determinism	☐ Deterministic ☑ Stochastic *(the same text can come from both kinds of site)*
Change over time	☑ Static ☐ Dynamic
State and actions	☑ Discrete ☐ Continuous
Agents	☑ Single-agent ☐ Multi-agent / adversarial
Agent type	☐ Simple reflex ☐ Model-based ☐ Goal-based ☐ Utility-based ☑ Learning

3.3 Problem Representation

Element	Your representation
State representation	Input features: the URL turned into (a) a vector of 30 numeric lexical features (URL length, '@', IP address, number of subdomains, prefix/suffix '-', shortener, etc.) and (b) a fixed-length sequence of character codes (planned length 200). Target to predict: label 1 = phishing, 0 = legitimate.
Actions / moves	Assign the label Phishing or Safe, attach a probability, and return the top contributing features as an explanation.
Initial state	A raw URL string submitted by the user, not yet parsed or scored.
Goal or terminal test	The URL has been scored and explained and the verdict returned; during evaluation, success means the predicted label equals the true label.
Cost, utility, fitness or evaluation	Training loss: Gini impurity (Random Forest) and binary cross-entropy (BiLSTM). Evaluation: accuracy, precision, recall, F1 and ROC-AUC. Ensemble score $P = w \times P_{RF} + (1 - w) \times P_{LSTM}$, flagged if $P \geq 0.5$.

4. Proposed AI Approach

#	Technique	Role in the project	Own code or library
1	Random Forest classifier	Core model: classifies a URL from its 30 engineered features.	Library (scikit-learn); own feature extraction and training script
2	Character-level Bidirectional LSTM	Second model: reads the raw URL characters to catch disguises such as paypa1.	Library (PyTorch); own model definition and training loop
3	Weighted ensemble	Fuses both probabilities into one verdict; weight tuned on validation data.	Own code
4	SHAP TreeExplainer	Explains each Random Forest prediction as top contributing features.	Library (shap); own plain-language mapping
B	**Baseline:** rule-based heuristic scorer	Counts red flags (length, '@', IP host, subdomains, '-') with fixed thresholds; the learned models are compared against it.	Own code

Phishing detection is a supervised classification problem with partial information: the agent sees only the text of the link, so it must learn which patterns tend to accompany malicious sites. A Random Forest suits the 30 numeric features because it is fast on a CPU, tolerates noisy features and works directly with SHAP, which lets us explain every verdict. A character-level BiLSTM reads the raw URL forwards and backwards, so it can catch disguises such as paypa1 that fixed features miss. The ensemble score is a weighted average of the two phishing probabilities, $P = w \times P_{RF} + (1 - w) \times P_{LSTM}$, starting at $w = 0.8$ and tuned on the validation set; a URL is flagged when $P \geq 0.5$. The rule-based baseline simply counts red flags (length, '@', IP host, subdomains) and shows how much learning adds.

4.1 Planned Class Design

- **DatasetManager** – attrs: raw_dir, cache_dir, seed; methods: download(), load_cached(), split(train/val/test)
- **URLFeatureExtractor** – attrs: feature_names (30); methods: extract(url) → vector, encode_chars(url, max_len), red_flags(url)
- **BaseDetector** (abstract) – methods: fit(X, y), predict_proba(url)

  - **RuleBasedDetector** (baseline) – attrs: rules, weights; scores red flags
  - **RandomForestDetector** – attrs: model (scikit-learn); methods: fit(), predict_proba(), save()/load()
  - **BiLSTMDetector** – attrs: vocab, network (PyTorch), epochs; methods: fit(), predict_proba(), save()/load()
- **EnsembleDetector** – has a RandomForestDetector and a BiLSTMDetector; attrs: weight w, threshold; methods: tune_weight(val), predict(url) → Verdict
- **ShapExplainer** – uses the RandomForestDetector; methods: explain(features, top_k) → list of (feature, impact, plain-text reason)
- **Evaluator** – uses any BaseDetector; methods: metrics(), confusion_matrix(), roc_curve(), time_inference(), plot_charts()
- **PhishGuardAPI** (FastAPI app) – endpoint POST /predict calls EnsembleDetector and ShapExplainer and returns JSON for the web page

5. Scope

In scope — the features we commit to delivering.

#	Feature	Why it is essential
1	URL feature extraction (30 features, string-only)	Gives both models their input and avoids slow network look-ups.
2	Random Forest and character-level BiLSTM trained on public data	These are the core learned models of the project.
3	Weighted ensemble giving verdict and probability	Combines both models into the single answer the user sees.
4	SHAP explanation of each verdict in plain words	Makes the prediction understandable to non-technical users.
5	Web interface (FastAPI back end, HTML/CSS/JS front end)	Lets a user paste a URL and read the result in a browser.
6	Evaluation against the baseline with at least three charts	Proves the AI works better than a simple rule program.

Out of scope — deliberately excluded. Paid or external LLM/API explanations (e.g. OpenAI), because paid sources are not permitted; fetching or inspecting live page content, WHOIS, DNS or screenshots; a browser extension or mobile app; scanning e-mails or SMS; transformer models; cloud deployment, user accounts or databases; continuous online retraining.

6. Data or Environment Source

Source name and URL	1) UCI Machine Learning Repository, “Phishing Websites” dataset: https://archive.ics.uci.edu/dataset/327/phishing+websites 2) A public Kaggle labelled phishing/legitimate URL dataset (kaggle.com), used for the character-level model and for testing our own extractor; the exact dataset is not decided yet and will be fixed in Week 11.
Type	Public dataset ☑ Web API ☐ Self-collected ☐ Generated / simulated ☐
Access requirements	No key needed ☐ Free key required ☑ *(UCI needs none; Kaggle needs a free account/token. No paid sources.)*
Approximate size	UCI: 11,055 labelled records with 30 features. Kaggle URL list: hundreds of thousands of URLs (a subset will be used if training is slow).
Licence / terms of use	UCI: Creative Commons Attribution 4.0 (CC BY 4.0), cited in the report. Kaggle: the licence stated on the chosen dataset page, checked before use.
Offline fallback	Datasets are downloaded once into data/raw/ and the processed copies saved in data/processed/ (CSV). Training and the app read only these local files, and the trained models are saved to disk, so the program runs with no network at all.

7. Evaluation Plan

What will be compared	Rule-based baseline vs Random Forest vs BiLSTM vs the RF + BiLSTM ensemble (same data, same split).
Metric(s)	Accuracy, precision, recall, F1, ROC-AUC, confusion matrix; average inference time per URL in seconds.
Baseline	Rule-based heuristic scorer (own code), described in Section 4.
Test instances / test set	One fixed stratified split of 70% train / 15% validation / 15% test, shared by all methods; plus a hand-made set of about 30 obfuscated URLs (paypa1, '@' tricks, hex IPs, long subdomain chains) as a robustness check.
Number of runs and seed	Split seed = 42; each learned model trained 5 times (seeds 42–46) and reported as mean ± standard deviation; latency timed over 100 URLs.
Planned charts (at least three)	(1) Bar chart of accuracy/precision/recall/F1 for all four methods; (2) confusion matrices; (3) ROC curves; (4) inference-time bar chart; (5) SHAP feature-importance summary plot.

8. Division of Work

Member	Modules / components owned	Approximate share
Sameer Danish (Leader)	Random Forest training, character-level BiLSTM training, ensemble fusion and weight tuning; repository management and final integration.	34%
Hadeer Abbas	Dataset download, offline cache and seeded split; 30-feature extractor and character encoder; rule-based baseline; evaluation scripts and charts.	33%
Muhammad Arham	FastAPI back end (/predict); SHAP explainer and plain-language reasons; web front end (HTML/CSS/JS); README, proposal and final report.	33%

9. Milestones and Timeline

#	Milestone — what will be demonstrably working	Target date	Owner
1	Dataset downloaded and cached locally; seeded train/val/test split saved; feature extractor returns 30 features for 20 sample URLs.	End of Week 11	Hadeer
2	Random Forest and rule baseline trained; script prints accuracy, precision, recall and F1 for both on the test set.	End of Week 12	Sameer, Hadeer
3	BiLSTM trained and ensemble working; POST /predict returns verdict and probability for any URL.	End of Week 13	Sameer, Arham
4	SHAP reasons and web page working: paste URL → verdict, probability bar and top reasons shown in under 3 s.	End of Week 14	Arham
5	All comparisons run with fixed seeds; at least three charts saved in the results folder.	End of Week 14	Hadeer
	Experiments complete, final integration, documentation and repository cleanup (README, proposal and report in docs/).	Week 15 (first week of Dec 2026)	All three

10. Expected Outcome and Risks

A working submission is a local web page where the user pastes a URL and, within three seconds, sees a Safe or Phishing verdict, a probability bar and the top reasons in plain words. We expect the Random Forest and the ensemble to reach about 85% or more accuracy on the held-out test set and to clearly beat the rule baseline. **Risk 1:** some UCI features need page or WHOIS data that a URL cannot give; we will keep only string-derived features and retrain. **Risk 2:** BiLSTM training may be too slow on laptop CPUs; we will use a data subset or a smaller network, and fall back to Random Forest plus rules if needed.

For office use — Lab Instructor	
Proposal status	Approved ☐ Approved with changes ☐ Revision required ☐
Comments	
Signature and date	

End of Project Proposal